# Лабораторна робота № 3

## Діагностика шкал вимірювання

Курс СК26 «Інтелектуальний аналіз та візуалізація даних»

**Тривалість:** 2 год аудиторно + 3 год самостійно · **Оцінка:** 4 бали **Файли:** `Ковід_дані_навчальні.xlsx`, `ЛР3_помилковий_аналіз.ipynb`

---

Шкала вимірювання визначає, які операції над змінною взагалі мають сенс. Комп'ютер цього не знає: він порахує середнє від номера телефону й не поскаржиться. Знати має вимірювач.

У цій роботі ви складаєте **паспорт восьми змінних** свого варіанта, показуєте на власних даних, які статистики для них недопустимі, і виправляєте чужий аналіз, де таких помилок десять.

```
ПРІЗВИЩЕ_ІМЯ = 'Перхун Максим'      # напишіть тут своє прізвище та ім'я
ГРУПА        = 'ШІ-34'
НОМЕР        = 11                    # ваш номер за списком групи (1–30)

assert ПРІЗВИЩЕ_ІМЯ and ГРУПА and НОМЕР, 'Заповніть три поля вище'
assert 1 <= НОМЕР <= 30, 'Номер має бути від 1 до 30'

import json, re
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
import warnings
warnings.filterwarnings('ignore')

pd.set_option('display.max_colwidth', None)     # не обрізати довгий текст у комірках

def показати(таблиця):
    # виводить таблицю з повним текстом: перенос на нові рядки, вирівнювання ліворуч
    return (таблиця.style.hide(axis='index')
            .set_properties(**{'text-align': 'left', 'white-space': 'normal', 'vertical-align': 'top'})
            .set_table_styles([{'selector': 'th', 'props': [('text-align', 'left')]}]))
pd.set_option('display.width', 200)
```

```
ВАРІАНТИ = json.load(open('lr3_variants_student.json', encoding='utf-
8'))
В = ВАРІАНТИ[НОМЕР - 1]

print('=' * 74)
print(f'ВАРІАНТ {В["варіант"]}   ·   {ПРІЗВИЩЕ_ІМЯ}   ·   {ГРУПА}')
print('=' * 74)
print()
print('Ваші вісім стовпців:')
for i, c in enumerate(В['стовпці'], 1):
    print(f'   {i}. {c}')
print()
print('Ваша пара для аналізу зв’язку:', ' та '.join(В['пара']))
print()
print('Ваше додаткове питання:')
print('  ', В['питання'])
```

```
==========================================================================
====
ВАРІАНТ 11   ·   Перхун Максим   ·   ШІ-34
==========================================================================
====

Ваші вісім стовпців:
   1. Телефон
   2. Примітка
   3. ЛН
   4. Виписка
   5. Ліжко днів
   6. Прокальцитонін
   7. Білок1
   8. Гематокрит2

Ваша пара для аналізу зв’язку: Вік та spO2 без дот

Ваше додаткове питання:
   Стовпець «Вік» можна перетворити на вікові групи «до 40», «40–60»,
«понад 60». Яка шкала була до перетворення і яка стала? Що ви при
цьому втратили і що здобули?
```

```
ФАЙЛ = 'Ковід_дані_навчальні.xlsx'
df = pd.read_excel(ФАЙЛ, sheet_name='Лист1', engine='openpyxl')
МОЇ = В['стовпці']

print('Розмір набору:', df.shape)
df[МОЇ].head(8)
```

```
Розмір набору: (121, 77)
```

```
                                    Телефон
Примітка      ЛН              Виписка   Ліжко днів  Прокальцитонін
Білок1  Гематокрит2
0                0958469788 дружина Ганна
NaN  III-IV  2022-03-10 00:00:00           14           0.126    70.0
45.0
1   0671778486, 0994887663 дружина Оксана   отримував актемра, часті
носові кровотечі, втрата нюху  II-III  2022-03-16 00:00:00
22             NaN    61.8         57.0
2   0958768045, чоловік Степан 0951784726                       часта
втрата свідомості в стаціонарі      II  2022-01-18 00:00:00
20             NaN    80.5         37.0
3  0959819331. дочка Валентина 0960845205
панкреонекроз  III-IV  2022-01-13 00:00:00           20
NaN    62.6         34.0
4     0952294866, дочка Дарина 0678360161
NaN  II-III  2022-03-11 00:00:00           27             NaN    72.0
48.0
5                 0959922983 сестра Софія
NaN  II-III  2021-12-26 00:00:00  9(5 у реан)             NaN    73.0
36.0
6  0998073702, дружина Наталія 0998382226  глюкозурія, кетонові тіла в
сечі, ЦД, фібриляція передс  II-III  2022-05-11 00:00:00           18
NaN    76.0         44.5
7       0637258771, син Сергій 0968413301
NaN       I  2022-05-28 00:00:00           10             NaN    88.0
42.0
```

---

# Етап 1. Паспорт змінних

Для кожного з восьми стовпців визначте шкалу вимірювання. Спирайтеся не на тип даних у pandas, а на **зміст величини**: що означають її значення і які порівняння між ними мають сенс.

Нагадування:

| Шкала | Ознака |
|---|---|
| Номінальна | значення можна лише порівняти на рівність |
| Порядкова | є природний порядок, але відстані між сусідніми рівнями невідомі |
| Інтервальна | різниці мають сенс, нуль умовний |
| Відношень | є природний нуль, відношення мають сенс |
| Абсолютна | природні і нуль, і одиниця вимірювання |

Окремо позначайте **ідентифікатори** — значення, що не є величиною взагалі (номер історії хвороби, прізвище, телефон).

```
# Спершу подивіться, що каже pandas — і не довіряйте йому
df[МОЇ].info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 121 entries, 0 to 120
Data columns (total 8 columns):
 #   Column          Non-Null Count  Dtype
---  ------          --------------  -----
 0   Телефон         120 non-null    object
 1   Примітка        45 non-null     str
 2   ЛН              121 non-null    str
 3   Виписка         120 non-null    object
 4   Ліжко днів      120 non-null    object
 5   Прокальцитонін  46 non-null     float64
 6   Білок1          118 non-null    float64
 7   Гематокрит2     101 non-null    float64
dtypes: float64(3), object(3), str(2)
memory usage: 7.7+ KB
```

```
# Далі — що є всередині насправді
for c in МОЇ:
    з = df[c].dropna()
    типи = ', '.join(f'{t.__name__}×{n}' for t, n in
з.map(type).value_counts().items())
    print(f'--- {c}')
    print(f'    непорожніх {len(з)} з {len(df)} | унікальних
{з.nunique()} | типи: {типи}')
    print(f'    приклади: {[repr(v) for v in з.unique()[:4]]}')
    print()
```

```
--- Телефон
    непорожніх 120 з 121 | унікальних 120 | типи: str×97, int×23
    приклади: ["'0958469788 дружина Ганна'", "'0671778486, 0994887663
дружина Оксана'", "'0958768045, чоловік Степан 0951784726'",
"'0959819331. дочка Валентина 0960845205'"]

--- Примітка
    непорожніх 45 з 121 | унікальних 44 | типи: str×45
    приклади: ["'отримував актемра, часті носові кровотечі, втрата
нюху'", "'часта втрата свідомості в стаціонарі'", "'панкреонекроз'",
"'глюкозурія, кетонові тіла в сечі, ЦД, фібриляція передс'"]

--- ЛН
    непорожніх 121 з 121 | унікальних 6 | типи: str×121
    приклади: ["'III-IV'", "'II-III'", "'II'", "'I'"]

--- Виписка
```

```
    непорожніх 120 з 121 | унікальних 90 | типи: datetime×118, str×2
    приклади: ['datetime.datetime(2022, 3, 10, 0, 0)',
'datetime.datetime(2022, 3, 16, 0, 0)', 'datetime.datetime(2022, 1,
18, 0, 0)', 'datetime.datetime(2022, 1, 13, 0, 0)']

--- Ліжко днів
    непорожніх 120 з 121 | унікальних 40 | типи: int×119, str×1
    приклади: ['14', '22', '20', '27']

--- Прокальцитонін
    непорожніх 46 з 121 | унікальних 17 | типи: float×46
    приклади: ['np.float64(0.126)', 'np.float64(0.9)',
'np.float64(0.1)', 'np.float64(0.104)']

--- Білок1
    непорожніх 118 з 121 | унікальних 44 | типи: float×118
    приклади: ['np.float64(70.0)', 'np.float64(61.8)',
'np.float64(80.5)', 'np.float64(62.6)']

--- Гематокрит2
    непорожніх 101 з 121 | унікальних 34 | типи: float×101
    приклади: ['np.float64(45.0)', 'np.float64(57.0)',
'np.float64(37.0)', 'np.float64(34.0)']
```

## Заповніть паспорт

Для кожного стовпця вкажіть шкалу та **обґрунтування** — одне речення про те, чому саме ця шкала. Обґрунтування оцінюється, назва шкали сама по собі — ні.

```
ПАСПОРТ = {
    'Телефон': ('ідентифікатор',
        'Це контактний номер родича, а не величина: усі 120 значень
унікальні, частина містить текст '
        '(«дружина Ганна») і кілька номерів; порядок і різниці між
номерами не мають жодного змісту.'),
    'Примітка': ('номінальна',
        'Вільний текст лікаря: записи можна лише порівняти на рівність
(«така сама / інша примітка»), '
        'порядку й відстаней між ними немає; порожня клітинка означає
«приміток немає».'),
    'ЛН': ('порядкова',
        'Ступінь легеневої недостатності (I < I-II < II < II-III < III
< III-IV) має природний клінічний порядок, '
        'але відстань між ступенями невідома й нерівна, тому середнє
для неї не має сенсу.'),
    'Виписка': ('інтервальна',
        'Календарна дата: різниця двох дат (тривалість у днях) має
сенс, але нуль шкали — початок '
```

```
        'літочислення — умовний, тому відношення дат («удвічі
пізніше») безглузде.'),
    'Ліжко днів': ('відношень',
        'Кількість днів у стаціонарі має природний нуль (0 днів = не
лежав), тому 28 днів справді '
        'вдвічі довше за 14; це лічильник, відношення значень мають
зміст.'),
    'Прокальцитонін': ('відношень',
        'Концентрація (нг/мл) має природний нуль — відсутність
речовини, — тож 10 нг/мл справді у 100 разів '
        'більше за 0.1; застереження: 0.1 — це нижня межа чутливості
тесту (цензуровані дані).'),
    'Білок1': ('відношень',
        'Концентрація загального білка в г/л має природний нуль та
одиницю вимірювання, тому '
        'відношення («удвічі більше білка») мають зміст і допустимі
всі статистики.'),
    'Гематокрит2': ('абсолютна',
        'Гематокрит — частка об’єму крові, яку займають еритроцити (у
%): безрозмірна величина, у якої і нуль, '
        'і одиниця (100 % = вся кров) задані природою, а не вибрані
дослідником.'),
}

for c in МОЇ:
    ПАСПОРТ.setdefault(c, ('', ''))

паспорт_df = pd.DataFrame(
    [(c, ПАСПОРТ[c][0], ПАСПОРТ[c][1]) for c in МОЇ],
    columns=['Стовпець', 'Шкала', 'Обґрунтування'])
показати(паспорт_df)
```

```
<pandas.io.formats.style.Styler at 0x1fbc521cad0>
```

```
# --- САМОПЕРЕВІРКА 1 ---
ДОПУСТИМІ = {'номінальна', 'порядкова', 'інтервальна', 'відношень',
             'абсолютна', 'ідентифікатор'}
for c in МОЇ:
    ш, об = ПАСПОРТ[c]
    assert ш.strip().lower() in ДОПУСТИМІ, f'{c}: шкалу «{ш}» не
розпізнано'
    assert len(об.strip()) >= 25, f'{c}: обґрунтування закоротке'
print('Самоперевірка 1 пройдена: усі вісім стовпців мають шкалу та
обґрунтування.')
```

```
Самоперевірка 1 пройдена: усі вісім стовпців мають шкалу та
обґрунтування.
```

---

# Етап 2. Допустимі статистики

Для кожного стовпця обчисліть **тільки те, що дозволяє його шкала**.

- номінальна та ідентифікатор — мода, частоти
- порядкова — додатково медіана та квантилі
- інтервальна — додатково середнє та стандартне відхилення
- відношень і абсолютна — усе, зокрема коефіцієнт варіації

Якщо для стовпця не можна порахувати нічого, крім частот, — так і напишіть.

## Допоміжний крок: виправляємо лише те, що можна виправити однозначно

Перш ніж рахувати статистики, треба переконатися, що дані придатні для цього. У моїх стовпцях є три дефекти, які видно з клітинок 5–6:

- **Ліжко днів** — одне значення записано текстом `'9(5 у реан)'`; беремо ціле число на початку (9 днів, з них 5 у реанімації);
- **Виписка** — два значення `'24.024.2021'` і `'20.24.2021'` не є датами (pandas їх не розпізнає). Відновлюємо їх як `Поступлення + Ліжко днів` і **перевіряємо** за незалежним стовпцем «Дата розтину» (обидва пацієнти померли);
- **ЛН** — ступінь `III-IV` написано латинськими `IV` серед кириличних `І`; порядок категорій задаємо вручну.

```
# --- Ліжко днів: '9(5 у реан)' -> 9 ---
def ліжко_днів(v):
    if pd.isna(v):
        return np.nan
    m = re.match(r'\s*(\d+)', str(v))
    return float(m.group(1)) if m else np.nan

ЛІЖКО = df['Ліжко днів'].map(ліжко_днів)

# --- Виписка: відновлюємо дві зіпсовані дати ---
ПОСТУПЛЕННЯ  = pd.to_datetime(df['Поступлення'], errors='coerce')
ВИПИСКА_СИРА = pd.to_datetime(df['Виписка'], errors='coerce')
зіпсовані = df.index[df['Виписка'].notna() & ВИПИСКА_СИРА.isna()]

ВИПИСКА = ВИПИСКА_СИРА.copy()
ВИПИСКА.loc[зіпсовані] = ПОСТУПЛЕННЯ.loc[зіпсовані] +
pd.to_timedelta(ЛІЖКО.loc[зіпсовані], unit='D')

перевірка = pd.DataFrame({
    'було': df.loc[зіпсовані, 'Виписка'],
    'відновлено': ВИПИСКА.loc[зіпсовані].dt.date,
    'дата розтину (незалежна)': pd.to_datetime(df.loc[зіпсовані, 'Дата
розтину']).dt.date,
})
```

```
print('Відновлення двох зіпсованих дат виписки (збіг із датою розтину підтверджує відновлення):')
print(перевірка.to_string())

# --- ЛН: кирилична «І» і латинська «I» — це різні символи; зводимо до одного алфавіту ---
ПОРЯДОК_ЛН = ['І', 'І-ІІ', 'ІІ', 'ІІ-ІІІ', 'ІІІ', 'ІІІ-IV']      # заданий вручну порядок
ЛН_лат = df['ЛН'].astype(str).str.strip().str.replace('I', 'І')
ЛН = pd.Series(pd.Categorical(ЛН_лат, categories=ПОРЯДОК_ЛН, ordered=True), index=df.index)
print()
print('ЛН: усі значення потрапили у задані категорії:', ЛН.notna().all())
print(ЛН.value_counts().sort_index().to_string())

# --- Зведений словник очищених стовпців для наступних клітинок ---
ЧИСТІ = {c: df[c] for c in МОЇ}
ЧИСТІ['Примітка']   = df['Примітка'].str.strip().replace('', np.nan)
ЧИСТІ['ЛН']         = ЛН
ЧИСТІ['Виписка']    = ВИПИСКА
ЧИСТІ['Ліжко днів'] = ЛІЖКО
```

```
Відновлення двох зіпсованих дат виписки (збіг із датою розтину підтверджує відновлення):
           було  відновлено дата розтину (незалежна)
74  24.024.2021  2022-04-24               2022-04-24
75   20.24.2021  2022-04-20               2022-04-20

ЛН: усі значення потрапили у задані категорії: True
І         12
І-ІІ       9
ІІ        21
ІІ-ІІІ    59
ІІІ       17
ІІІ-IV     3
```

```
# Чому не можна довіряти автоматичному сортуванню римських цифр
print('Сортування рядків як є:  ', sorted(df['ЛН'].unique()))
print('Задано вручну:           ', [x for x in ['І', 'І-ІІ', 'ІІ', 'ІІ-ІІІ', 'ІІІ', 'ІІІ-IV']])
print()
print('Збіг випадковий. Достатньо двох типових відхилень запису:')
print('  латинські I замість кириличних І :', sorted(['ІІ', 'ІІІ', 'I']), ' <- «ІІІ» опинилось перед «I»')
print('  ступінь IV (латиниця)            :', sorted(['ІІІ', 'IV', 'ІІ']), ' <- IV стало найменшим')
print('Кирилична «І» (U+0406) і латинська «I» (U+0049) — різні символи; у даних є обидві:',
```

```
      sorted({hex(ord(ch)) for v in df['ЛН'].unique() for ch in v if
ch != '-'}))

Сортування рядків як є:   ['I', 'I-II', 'II', 'II-III', 'III', 'III-
IV']
Задано вручну:            ['I', 'I-II', 'II', 'II-III', 'III', 'III-
IV']

Збіг випадковий. Достатньо двох типових відхилень запису:
  латинські I замість кириличних I : ['III', 'I', 'II']  <- «III»
опинилось перед «I»
  ступінь IV (латиниця)           : ['IV', 'II', 'III']  <- IV стало
найменшим
Кирилична «I» (U+0406) і латинська «I» (U+0049) — різні символи; у
даних є обидві: ['0x406', '0x49', '0x56']

def допустима_статистика(стовпець, шкала):
    """Повертає словник статистик, дозволених для цієї шкали."""
    з = ЧИСТІ[стовпець].dropna()
    р = {'непорожніх': len(з), 'унікальних': з.nunique()}
    ш = шкала.strip().lower()

    if ш in ('ідентифікатор',):
        # ідентифікатор: лише контроль якості (повнота, унікальність);
частоти — не для аналізу
        р['дублікатів'] = int(з.duplicated().sum())
        return р

    # --- усі шкали: мода і частоти ---
    частоти = з.value_counts()
    р['мода (значення, частота)'] = (частоти.index[0],
int(частоти.iloc[0]))
    if ш == 'номінальна':
        return р

    # --- порядкова: + медіана та квартилі (за рангом категорії, не за
значенням) ---
    if ш == 'порядкова':
        кат = list(з.cat.categories)
        коди = np.sort(з.cat.codes.values)
        def квантиль(р):                        # метод найближчого
рангу
            return кат[коди[int(np.ceil(р * len(коди))) - 1]]
        р['медіана'] = квантиль(0.5)
        р['Q1 .. Q3'] = (квантиль(0.25), квантиль(0.75))
        return р

    # --- інтервальна (дати): + середнє, SD, різниці; відношення й CV
заборонені ---
    if ш == 'інтервальна':
```

```
        p['мін .. макс'] = (з.min().date(), з.max().date())
        p['медіана'] = з.median().date()
        p['Q1 .. Q3'] = (з.quantile(.25).date(),
з.quantile(.75).date())
        p['середня дата'] = з.mean().date()
        p['SD, днів'] = round(з.std() / pd.Timedelta(days=1), 1)
        p['розмах, днів'] = (з.max() - з.min()).days
        return p

    # --- відношень / абсолютна: + CV, геометричне середнє ---
    if ш in ('відношень', 'абсолютна'):
        x = з.astype(float)
        p['мін .. макс'] = (round(x.min(), 3), round(x.max(), 3))
        p['медіана'] = round(x.median(), 3)
        p['Q1 .. Q3'] = (round(x.quantile(.25), 3),
round(x.quantile(.75), 3))
        p['середнє'] = round(x.mean(), 3)
        p['SD'] = round(x.std(), 3)
        p['CV, %'] = round(100 * x.std() / x.mean(), 1)
        додатні = x[x > 0]
        p['геометричне середнє'] =
round(float(np.exp(np.log(додатні).mean())), 3)
        return p
    return p


for c in МОЇ:
    print(f'--- {c}  ({ПАСПОРТ[c][0]})')
    print('   ', допустима_статистика(c, ПАСПОРТ[c][0]))
    print()
```

```
--- Телефон  (ідентифікатор)
    {'непорожніх': 120, 'унікальних': 120, 'дублікатів': 0}

--- Примітка  (номінальна)
    {'непорожніх': 44, 'унікальних': 42, 'мода (значення, частота)':
('Поліалергетик', 2)}

--- ЛН  (порядкова)
    {'непорожніх': 121, 'унікальних': 6, 'мода (значення, частота)':
('II-III', 59), 'медіана': 'II-III', 'Q1 .. Q3': ('II', 'II-III')}

--- Виписка  (інтервальна)
    {'непорожніх': 120, 'унікальних': 88, 'мода (значення, частота)':
(Timestamp('2021-11-19 00:00:00'), 3), 'мін .. макс':
(datetime.date(2021, 4, 27), datetime.date(2022, 6, 22)), 'медіана':
datetime.date(2022, 3, 20), 'Q1 .. Q3': (datetime.date(2021, 11, 18),
datetime.date(2022, 4, 27)), 'середня дата': datetime.date(2022, 1,
31), 'SD, днів': 109.1, 'розмах, днів': 421}
```

```
--- Ліжко днів  (відношень)
    {'непорожніх': 120, 'унікальних': 39, 'мода (значення, частота)':
(np.float64(10.0), 12), 'мін .. макс': (np.float64(1.0),
np.float64(79.0)), 'медіана': np.float64(15.0), 'Q1 .. Q3':
(np.float64(10.0), np.float64(22.0)), 'середнє': np.float64(17.883),
'SD': np.float64(12.937), 'CV, %': np.float64(72.3), 'геометричне
середнє': 14.156}

--- Прокальцитонін  (відношень)
    {'непорожніх': 46, 'унікальних': 17, 'мода (значення, частота)':
(np.float64(0.1), 30), 'мін .. макс': (np.float64(0.043),
np.float64(18.22)), 'медіана': np.float64(0.1), 'Q1 .. Q3':
(np.float64(0.1), np.float64(0.12)), 'середнє': np.float64(0.753),
'SD': np.float64(3.01), 'CV, %': np.float64(399.8), 'геометричне
середнє': 0.145}

--- Білок1  (відношень)
    {'непорожніх': 118, 'унікальних': 44, 'мода (значення, частота)':
(np.float64(70.0), 12), 'мін .. макс': (np.float64(50.0),
np.float64(88.0)), 'медіана': np.float64(66.0), 'Q1 .. Q3':
(np.float64(60.0), np.float64(72.0)), 'середнє': np.float64(66.753),
'SD': np.float64(8.378), 'CV, %': np.float64(12.6), 'геометричне
середнє': 66.241}

--- Гематокрит2  (абсолютна)
    {'непорожніх': 101, 'унікальних': 34, 'мода (значення, частота)':
(np.float64(40.0), 15), 'мін .. макс': (np.float64(16.0),
np.float64(57.0)), 'медіана': np.float64(40.0), 'Q1 .. Q3':
(np.float64(36.0), np.float64(46.0)), 'середнє': np.float64(41.141),
'SD': np.float64(7.566), 'CV, %': np.float64(18.4), 'геометричне
середнє': 40.375}
```

**Що тут свідомо не обчислюється:**

- **Телефон** — тільки повнота й унікальність. Мода й частоти формально допустимі, але всі 120 значень унікальні, тож вони нічого не кажуть.
- **Примітка** — лише мода й частоти (мода тут — випадковий збіг двох однакових записів).
- **ЛН** — медіана й квартилі, але **не** середнє: «середній ступінь 3.4» не існує, бо відстані між ступенями невідомі.
- **Виписка** — медіана й квартили (успадковано від порядкової), середня дата, SD у днях та розмах допустимі (різниці дат мають зміст), але **коефіцієнт варіації заборонений**: нуль шкали умовний.
- **Прокальцитонін** — формально допустимі всі статистики, але 30 з 46 значень дорівнюють 0.1 (нижня межа чутливості тесту), тому середнє й SD описують не пацієнтів, а склад цензурованих даних. Надійні тут — медіана, квартилі та частка значень вище порогу.

- **Гематокрит2** — абсолютна шкала: допустимі всі статистики; пам'ятаємо лише, що 20 значень відсутні.

---

# Етап 3. Три безглузді числа

`df.describe()` порахує статистики для всього, що схоже на число. Частина цих чисел не має жодного змісту.

Знайдіть **щонайменше три** такі числа у своєму наборі: обчисліть їх, а потім поясніть, чому кожне з них безглузде.

```
df[МОЇ].describe(include='all').T
```

```
                count unique                         top  freq
mean       std    min   25%   50%     75%    max
Телефон           120    120  0958469788 дружина Ганна     1
NaN       NaN    NaN   NaN   NaN     NaN    NaN
Примітка           45     44             Поліалергетик     2
NaN       NaN    NaN   NaN   NaN     NaN    NaN
ЛН                121      6                   II-III    59
NaN       NaN    NaN   NaN   NaN     NaN    NaN
Виписка           120     90       2021-11-19 00:00:00     3
NaN       NaN    NaN   NaN   NaN     NaN    NaN
Ліжко днів      120.0   40.0                      10.0  12.0
NaN       NaN    NaN   NaN   NaN     NaN    NaN
Прокальцитонін   46.0    NaN                       NaN   NaN
0.752957  3.010375  0.043   0.1   0.1  0.1205  18.22
Білок1          118.0    NaN                       NaN   NaN
66.75339   8.37849   50.0  60.0  66.0    72.0   88.0
Гематокрит2     101.0    NaN                       NaN   NaN
41.140594  7.565992   16.0  36.0  40.0    46.0   57.0
```

```
# 1. Телефон: describe() середнє НЕ показав (стовпець змішаний: числа
+ текст), але варто
#    лише привести його до числа — і pandas порахує без жодних
заперечень.
тел_числа = pd.to_numeric(df['Телефон'], errors='coerce').dropna()
# лишилось 23 з 120 (решта — текст)

# 2. ЛН: «середній ступінь» залежить від довільного кодування
порядкових категорій
к_ранги = ЛН.cat.codes + 1                                    #
1,2,3,4,5,6
к_римські = ЛН.map({'I': 1, 'I-II': 1.5, 'II': 2, 'II-III': 2.5,
'III': 3, 'III-IV': 3.5}).astype(float)
к_клін = ЛН.map({'I': 1, 'I-II': 2, 'II': 3, 'II-III': 4, 'III': 5,
'III-IV': 10}).astype(float)
```

```
def назва_ступеня(середнє, коди):
    пара = pd.DataFrame({'к': коди.values, 'л':
ЛН.values}).drop_duplicates().sort_values('к')
    return пара.iloc[(пара['к'] - середнє).abs().argmin()]['л']
сер = [(к.mean(), назва_ступеня(к.mean(), к)) for к in (к_ранги,
к_римські, к_клін)]

# 3. Виписка: коефіцієнт варіації дат залежить від вибору «нуля»
def cv_дат(початок):
    дні = (ВИПИСКА.dropna() - pd.Timestamp(початок)).dt.days
    return 100 * дні.std() / дні.mean()
cv_1970, cv_2021 = cv_дат('1970-01-01'), cv_дат('2021-01-01')

# 4. Примітка: «найчастіше значення» вільного тексту
п = ЧИСТІ['Примітка'].dropna(); п_мода = п.value_counts()

# 5. Прокальцитонін: середнє цензурованих даних
пкт = df['Прокальцитонін'].dropna()
частка_нижче_середнього = (пкт < пкт.mean()).mean()

БЕЗГЛУЗДІ = [
    ('Телефон', 'середнє після to_numeric (describe його не рахує)',
round(тел_числа.mean(), 0),
     f'Номер — ярлик, а не величина: «середній телефон»
{тел_числа.mean():,.0f} нікому не належить. '
     'До того ж у цих 23 номерах зник початковий нуль (9 цифр замість
10), тож вони навіть не порівнювані з рештою.'),
    ('Телефон', 'SD після to_numeric', round(тел_числа.std(), 0),
     'Розкид номерів не вимірює нічого: різниця двох номерів не є
величиною, а від кодів операторів залежить лише розташування в
нумерації.'),
    ('ЛН', 'середній код ступеня (кодування 1..6)', round(сер[0][0],
2),
     f'Середнє для порядкової шкали залежить від довільних кодів: ті
самі дані дають «ступінь» {сер[0][1]} при кодах 1..6, '
     f'{сер[1][1]} при римських значеннях і {сер[2][1]} при кодах
1,2,3,4,5,10 — хоча порядок не змінився.'),
    ('ЛН', 'середній код ступеня (кодування 1,2,3,4,5,10)',
round(сер[2][0], 2),
     'Те саме питання, інше монотонне кодування — інше середнє.
Результат, який змінюється від вибору підписів, не є властивістю
даних.'),
    ('Виписка', 'коефіцієнт варіації дат (відлік від 1970)',
round(cv_1970, 3),
     f'CV = SD / середнє, а середнє дат залежить від умовного нуля:
той самий розкид дає {cv_1970:.2f} % при відліку від 1970 і '
     f'{cv_2021:.1f} % при відліку від 2021. Для інтервальної шкали
відношення не визначені.'),
    ('Примітка', 'найчастіше значення (describe: top/freq)',
f'«{п_мода.index[0]}» ×{п_мода.iloc[0]}',
```

```
     'Примітка — вільний текст. Два однакових записи з 44 — це збіг формулювань, а не «типова» примітка; жодна частота не описує пацієнтів.'),
    ('Прокальцитонін', 'середнє (усі непорожні)', round(пкт.mean(), 3),
     f'30 з {len(пкт)} значень дорівнюють 0.1 — це нижня межа чутливості тесту, а не вимірювання, і ще два значення 10 та 18.22 тягнуть вправо: '
     f'середнє більше за {100 * частка_нижче_середнього:.0f} % спостережень, а SD ({пкт.std():.2f}) вчетверо більша за середнє. Типового пацієнта воно не описує.'),
]

assert len(БЕЗГЛУЗДІ) >= 3, 'Потрібно щонайменше три приклади'
показати(pd.DataFrame(БЕЗГЛУЗДІ, columns=['Стовпець', 'Статистика', 'Значення', 'Чому безглузде']).astype({'Значення': str}))
```

```
<pandas.io.formats.style.Styler at 0x1fbc5200550>
```

**Звідки ці числа.** `describe()` показав лише частину з них (середнє Прокальцитоніну, `top/freq` для Примітки та Телефону). Решту я обчислив вручну саме для того, щоб показати: щойно дані приведено до числа або дати, pandas рахує їх без заперечень (для Телефону describe мовчить лише тому, що в стовпці лишився текст, — це випадковість, а не захист).

**Чому pandas не скаржиться.** Бібліотека знає лише *тип даних* (число, текст, дата), але не *шкалу* — вона не може відрізнити номер телефону від маси тіла, якщо обидва записані цілими числами. Формула середнього працює над будь-якими числами; питання «чи є результат величиною» поставити може лише людина.

---

# Етап 4. Міра зв'язку для вашої пари

Для двох змінних вашого варіанта треба оцінити зв'язок. Міру обирає **пара шкал**, а не зручність:

| Поєднання шкал | Міра |
|---|---|
| відношень × відношень | кореляція Пірсона |
| порядкова × порядкова | рангова кореляція Спірмена |
| номінальна × номінальна | $\chi^2$ та коефіцієнт Крамера (V) |
| відношень × бінарна номінальна | точковий бісеріальний коефіцієнт |
| номінальна × порядкова | коефіцієнт Крамера (V) |

Обґрунтуйте вибір, обчисліть міру й поясніть результат. Окремо покажіть, **що станеться**, якщо застосувати Пірсона там, де він недопустимий, — і чому отримане число не можна інтерпретувати.

```
A, B = B['пара']
print('Ваша пара:', A, '×', B)
print()
print(f'{A}: шкала —', ПАСПОРТ.get(A, ('не у вашому списку',))[0] or
'(визначте самостійно)')
print(f'{B}: шкала —', ПАСПОРТ.get(B, ('не у вашому списку',))[0] or
'(визначте самостійно)')
print()
print(df[[A, B]].head(6).to_string())
```

```
Ваша пара: Вік × spO2 без дот

Вік: шкала — не у вашому списку
spO2 без дот: шкала — не у вашому списку

   Вік  spO2 без дот
0   55          90.0
1   43          83.0
2   54          78.0
3   68          82.0
4   65          80.0
5   66          72.0
```

```
# Шкали пари (Вік і spO2 не входять у мої вісім стовпців, тож визначаю
їх тут окремо)
ПАСПОРТ_ПАРИ = {
    'Вік': ('відношень', 'Повні роки життя: природний нуль
(народження), 60 років справді вдвічі більше за 30.'),
    'spO2 без дот': ('абсолютна', 'Частка гемоглобіну, насиченого
киснем (%): нуль і 100 % задані природою.'),
}
for к, (ш, о) in ПАСПОРТ_ПАРИ.items():
    print(f'{к:14s} — {ш:10s} | {о}')
print()
print('Пара: відношень × абсолютна — обидві шкали вищі за інтервальну,
тож допустима кореляція Пірсона.')
print()
print('Дефект у spO2: мінімум', df['spO2 без дот'].min(), '(наступне
за величиною —',
      sorted(df['spO2 без дот'].dropna())[1], '). Значення 0.9 — це 90
%, записане часткою (як у стовпці КТ).')
```

```
Вік            — відношень  | Повні роки життя: природний нуль
(народження), 60 років справді вдвічі більше за 30.
spO2 без дот   — абсолютна  | Частка гемоглобіну, насиченого киснем
(%): нуль і 100 % задані природою.

Пара: відношень × абсолютна — обидві шкали вищі за інтервальну, тож
допустима кореляція Пірсона.
```

```
Дефект у spO2: мінімум 0.9 (наступне за величиною — 50.0 ). Значення
0.9 — це 90 %, записане часткою (як у стовпці КТ).

ОБРАНА_МІРА = 'Кореляція Пірсона (r) з перевіркою рангової кореляцією
Спірмена'
ЧОМУ_САМЕ_ВОНА = ('Вік (відношень) і spO2 (абсолютна) — обидві
кількісні шкали з осмисленими різницями, тому '
                  'допустима лінійна кореляція Пірсона; Спірмена додаю
як перевірку стійкості, бо spO2 має стелю біля 98 % '
                  'і несиметричний розподіл.')

assert ОБРАНА_МІРА and len(ЧОМУ_САМЕ_ВОНА) >= 40, 'Заповніть обидва
поля'

вік = df['Вік'].astype(float)
сат_сира = df['spO2 без дот']
сат = сат_сира.where(сат_сира != 0.9, 90.0)        # 0.9 -> 90 (частка
записана замість відсотка)

def пірсон(x, y):
    m = pd.concat([x, y], axis=1).dropna()
    r, p = stats.pearsonr(m.iloc[:, 0], m.iloc[:, 1])
    n = len(m); z = np.arctanh(r); se = 1 / np.sqrt(n - 3)
    lo, hi = np.tanh([z - 1.96 * se, z + 1.96 * se])
    rho, p_s = stats.spearmanr(m.iloc[:, 0], m.iloc[:, 1])
    return n, r, p, lo, hi, rho, p_s

варіанти = {
    'сирі дані (з 0.9 як є)':          (вік, сат_сира),
    'виправлено: 0.9 -> 90  [основне]': (вік, сат),
    'без запису 0.9':                  (вік, сат_сира.where(сат_сира !
= 0.9)),
}
рядки = []
for назва, (x, y) in варіанти.items():
    n, r, p, lo, hi, rho, p_s = пірсон(x, y)
    рядки.append((назва, n, round(r, 3), f'[{lo:.2f}; {hi:.2f}]',
f'{p:.4f}', round(rho, 3), f'{p_s:.4f}'))
табл = pd.DataFrame(рядки, columns=['версія даних', 'n', 'Пірсон r',
'95% ДІ', 'p (Пірсон)', 'Спірмен ρ', 'p (Спірмен)'])
print(табл.to_string(index=False))

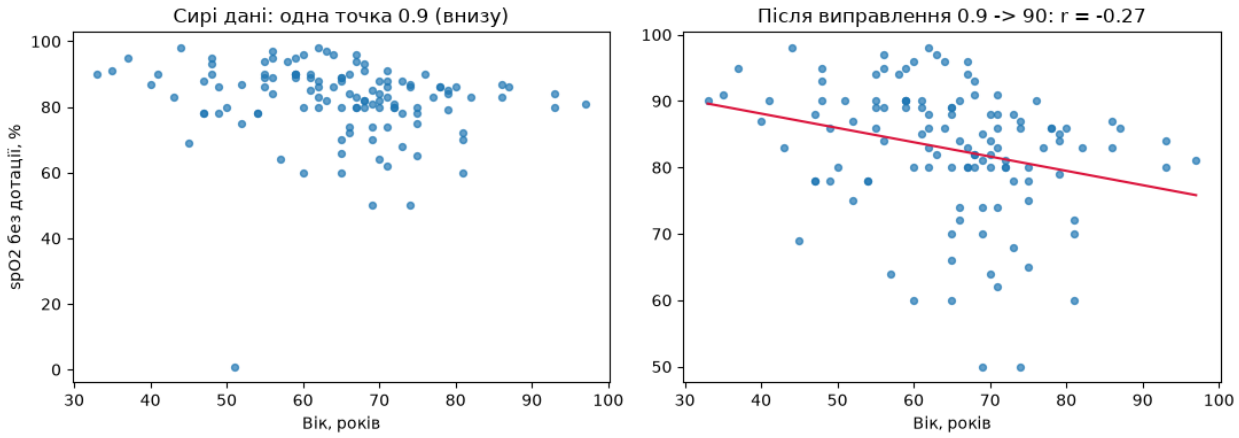
n, r, p, lo, hi, rho, p_s = пірсон(вік, сат)
print()
r_осн, p_осн = r, p
print(f'Основний результат: r = {r:.3f} (95% ДІ [{lo:.2f}; {hi:.2f}]),
p = {p:.4f}, n = {n}; R² = {r**2:.3f}.')
print(f'Спірмен ρ = {rho:.3f} — той самий знак і схожа сила, тож
висновок не залежить від форми розподілу.')
print('Інтерпретація: слабкий-помірний негативний зв’язок — з віком
```

```
сатурація при поступленні в середньому нижча;')
print(f'вік пояснює лише ~{100 * r**2:.0f} % варіації spO2; зв’язок не
доводить причинності (вік пов’язаний із супутніми хворобами).')

fig, ax = plt.subplots(1, 2, figsize=(11, 4))
ax[0].scatter(вік, сат_сира, s=18, alpha=.7)
ax[0].set_title('Сирі дані: одна точка 0.9 (внизу)');
ax[0].set_xlabel('Вік, років'); ax[0].set_ylabel('spO2 без дотації,
%')
m = pd.concat([вік, сат], axis=1).dropna()
ax[1].scatter(m.iloc[:, 0], m.iloc[:, 1], s=18, alpha=.7)
k, b = np.polyfit(m.iloc[:, 0], m.iloc[:, 1], 1)
xs = np.linspace(m.iloc[:, 0].min(), m.iloc[:, 0].max(), 50)
ax[1].plot(xs, k * xs + b, color='crimson')
ax[1].set_title(f'Після виправлення 0.9 -> 90: r = {r:.2f}');
ax[1].set_xlabel('Вік, років')
plt.tight_layout(); plt.show()
```

```
                    версія даних   n  Пірсон r          95% ДІ p
(Пірсон)  Спірмен ρ p (Спірмен)
          сирі дані (з 0.9 як є) 120    -0.146  [-0.32; 0.03]
0.1107     -0.309      0.0006
виправлено: 0.9 -> 90  [основне] 120    -0.271 [-0.43; -0.10]
0.0027     -0.342      0.0001
                  без запису 0.9 119    -0.266 [-0.43; -0.09]
0.0034     -0.334      0.0002
```

```
Основний результат: r = -0.271 (95% ДІ [-0.43; -0.10]), p = 0.0027, n
= 120; R² = 0.074.
Спірмен ρ = -0.342 — той самий знак і схожа сила, тож висновок не
залежить від форми розподілу.
Інтерпретація: слабкий-помірний негативний зв’язок — з віком сатурація
при поступленні в середньому нижча;
вік пояснює лише ~7 % варіації spO2; зв’язок не доводить причинності
(вік пов’язаний із супутніми хворобами).
```



```
# Контрприклад: Пірсон там, де він недопустимий.
# Правило перевірки: результат кореляції не повинен змінюватися від
того, як ми ПІДПИСАЛИ категорії.

# (а) Вік × ЛН (порядкова) — два однаково «правильні» монотонні
кодування дають різні r
м = pd.DataFrame({'вік': вік, 'А': ЛН.cat.codes + 1,
                  'Б': ЛН.map({'I': 1, 'I-II': 2, 'II': 3, 'II-III':
4, 'III': 5, 'III-IV': 10}).astype(float)})
r_а = м['вік'].corr(м['А']); r_б = м['вік'].corr(м['Б'])

# (б) Стать × помер/жив (номінальні) — r міняє знак, якщо поміняти
місцями 0 і 1
стать = df['Стать'].str.strip().str.lower().str[0]
помер = (df['помер/жив'] == 1).astype(int)
r_1 = стать.map({'ч': 0, 'ж': 1}).corr(помер)
r_2 = стать.map({'ч': 1, 'ж': 0}).corr(помер)

# (в) Вік × Телефон (ідентифікатор) — число рахується, змісту немає
т = df['Телефон'].map(lambda v: float(v) if isinstance(v, (int,
np.integer)) else np.nan)
r_в = вік.corr(т); n_в = pd.concat([вік, т], axis=1).dropna().shape[0]

print(pd.DataFrame([
    ('Вік × ЛН, коди 1..6',            round(r_а, 3), 'порядкова,
довільні коди'),
    ('Вік × ЛН, коди 1,2,3,4,5,10',    round(r_б, 3), 'той самий
порядок — інше r'),
    ('Стать(ч=0,ж=1) × летальність',   round(r_1, 3), 'номінальна,
довільні коди'),
    ('Стать(ч=1,ж=0) × летальність',   round(r_2, 3), 'знак змінився
від перейменування'),
    (f'Вік × Телефон (n = {n_в})',      round(r_в, 3), 'ідентифікатор,
коди операторів'),
], columns=['пара', 'Пірсон r', 'що не так']).to_string(index=False))
print()
print('Висновок: число r змінюється від підписів, які ми дали
категоріям, а не від даних. Пірсон припускає, що різниці')
print('між значеннями рівні й мають зміст. Для порядкової шкали це не
так (допустимий Спірмен), для номінальної —')
print('використовують χ² і V Крамера, для ідентифікатора — нічого.')
```

```
                        пара  Пірсон r                          що не
так
         Вік × ЛН, коди 1..6     0.398          порядкова, довільні
коди
 Вік × ЛН, коди 1,2,3,4,5,10     0.302       той самий порядок — інше
r
Стать(ч=0,ж=1) × летальність     0.177         номінальна, довільні
коди
```

```
Стать(ч=1,ж=0) × летальність    -0.177 знак змінився від перейменування
      Вік × Телефон (n = 23)    -0.238   ідентифікатор, коди операторів

Висновок: число r змінюється від підписів, які ми дали категоріям, а не від даних. Пірсон припускає, що різниці
між значеннями рівні й мають зміст. Для порядкової шкали це не так (допустимий Спірмен), для номінальної —
використовують χ² і V Крамера, для ідентифікатора — нічого.
```

---

# Етап 5. Підготовка змінних до моделі

Шкала визначає не лише статистику, а й спосіб кодування.

| Шкала | Як готують до моделі |
|---|---|
| Номінальна | one-hot кодування; порядок категорій не вводять |
| Порядкова | впорядкована категорія або цілі коди із **заданим** порядком |
| Інтервальна | стандартизація; для дат — перехід до тривалостей |
| Відношень | нормалізація або стандартизація; можливе логарифмування |
| Ідентифікатор | у модель не беруть узагалі |

Для кожного свого стовпця вкажіть спосіб підготовки — і **виконайте його** щонайменше для трьох стовпців різних шкал.

```
ПІДГОТОВКА = {
    'Телефон': 'У модель не беремо взагалі: ідентифікатор (і персональні дані); корисно лише як ключ запису.',
    'Примітка': 'Номінальна текстова: бінарна ознака «є примітка» + one-hot прапорці за ключовими словами (ЦД, кровотеча, інфаркт); порядок не вводимо.',
    'ЛН': 'Порядкова: цілі коди 0..5 із заданим вручну порядком I < I-II < II < II-III < III < III-IV (не алфавітним), без стандартизації як «відстані».',
    'Виписка': 'Інтервальна (дата): не подаємо як дату, а переходимо до тривалості Виписка − Поступлення в днях, далі стандартизація.',
    'Ліжко днів': 'Відношень, правий хвіст (до 79): розбір тексту «9(5 у реан)», логарифм log(1+x), потім стандартизація.',
    'Прокальцитонін': 'Відношень із цензурою 0.1: прапорці «виміряно» і «≤ 0.1», логарифм (хвіст до 18), стандартизація; пропуски = «не
```

```
призначали».',
    'Білок1': 'Відношень: стандартизація (z-оцінка); 3 пропуски
заповнюємо медіаною, обчисленою лише на навчальній вибірці.',
    'Гематокрит2': 'Абсолютна (частка): ділимо на 100, щоб лишитися на
природній шкалі [0; 1]; 20 пропусків — медіана + прапорець пропуску.',
}
for c in МОЇ:
    ПІДГОТОВКА.setdefault(c, '')

показати(pd.DataFrame([(c, ПАСПОРТ[c][0], ПІДГОТОВКА[c]) for c in
МОЇ],
                      columns=['Стовпець', 'Шкала', 'Як готуємо']))
```

```
<pandas.io.formats.style.Styler at 0x1fbc75765d0>
```

```
# --- Захисні функції: нормалізація тільки після перевірки
варіативності ---
def перевірити_варіативність(s, назва):
    з = pd.Series(s).dropna()
    if len(з) < 2 or з.nunique() == 1:
        raise ValueError(f'«{назва}»: сталий або майже порожній
стовпець — max = min, знаменник = 0')
    return з

def мінмакс(s, назва):
    з = перевірити_варіативність(s, назва)
    return (s - з.min()) / (з.max() - з.min())

def стандартизувати(s, назва):
    з = перевірити_варіативність(s, назва)
    return (s - з.mean()) / з.std()

# Демонстрація: сталий стовпець ФЕРИТ2 (єдине значення 1265)
try:
    мінмакс(df['ФЕРИТ2'], 'ФЕРИТ2')
except ValueError as e:
    print('Функція зупинила нормалізацію:', e)
з_ = df['ФЕРИТ2'].dropna()
print('Без захисту: (x - min)/(max - min) =', ((з_ - з_.min()) /
(з_.max() - з_.min())).tolist(), '-> NaN (0/0)')
print()

# --- Підготовка всіх моїх змінних (крім ідентифікатора) ---
X = pd.DataFrame(index=df.index)

# ЛН (порядкова): цілі коди із заданим порядком
X['ЛН_код'] = ЛН.cat.codes

# Примітка (номінальна): one-hot прапорці
текст = df['Примітка'].fillna('').str.lower()
```

```
X['примітка_є'] = (текст.str.len() > 0).astype(int)
for назва, шаблон in {'примітка_ЦД_глюкозурія': r'цд|глюкоз|цукров',
'примітка_кровотеча': r'кровотеч',
                      'примітка_інфаркт_стент': r'інфаркт|інфарк|
стент'}.items():
    X[назва] = текст.str.contains(шаблон).astype(int)

# Виписка (інтервальна): перехід до тривалості; записи, де дати
суперечать «Ліжко днів» (різниця > 3 дні), ставимо в NaN
трив = (ВИПИСКА - ПОСТУПЛЕННЯ).dt.days.astype(float)
некоректні = (трив - ЛІЖКО).abs() > 3
print('Тривалість: записів із суперечливими датами (заміна на NaN):',
int(некоректні.sum()), '| індекси:', list(трив.index[некоректні]))
трив = трив.where(~некоректні)
X['тривалість_z'] = стандартизувати(трив, 'тривалість').fillna(0)

# Ліжко днів (відношень): log(1+x) + стандартизація
X['ліжко_лог_z'] = стандартизувати(np.log1p(ЛІЖКО), 'Ліжко
днів').fillna(0)

# Прокальцитонін (відношень): прапорці + логарифм + стандартизація
пкт = df['Прокальцитонін']
X['ПКТ_виміряно'] = пкт.notna().astype(int)
X['ПКТ_≤0.1'] = (пкт <= 0.1).astype(int)
X['ПКТ_лог_z'] = стандартизувати(np.log10(пкт.fillna(0.1)),
'Прокальцитонін')

# Білок1 (відношень): стандартизація, пропуски — медіана
білок = df['Білок1'].fillna(df['Білок1'].median())
X['білок_z'] = стандартизувати(білок, 'Білок1')

# Гематокрит2 (абсолютна): частка [0;1], пропуски — медіана +
прапорець
гк = df['Гематокрит2']
X['гематокрит_частка'] = (гк / 100).fillna(гк.median() / 100)
X['гематокрит_пропуск'] = гк.isna().astype(int)

assert 'Телефон' not in X.columns          # ідентифікатор у модель не
беремо
assert X.isna().sum().sum() == 0, 'У матриці ознак лишилися NaN'
assert (X.nunique() > 1).all(), 'У матриці ознак є сталі стовпці'
print()
print('Матриця ознак:', X.shape)
X.describe().T[['mean', 'std', 'min', 'max']].round(3)
```

```
Функція зупинила нормалізацію: «ФЕРИТ2»: сталий або майже порожній
стовпець — max = min, знаменник = 0
Без захисту: (x - min)/(max - min) = [nan] -> NaN (0/0)

Тривалість: записів із суперечливими датами (заміна на NaN): 5 |
```

```
індекси: [40, 53, 59, 77, 119]

Матриця ознак: (121, 13)

                          mean    std    min    max
ЛН_код                   2.570  1.196  0.000  5.000
примітка_є               0.372  0.485  0.000  1.000
примітка_ЦД_глюкозурія   0.050  0.218  0.000  1.000
примітка_кровотеча       0.041  0.200  0.000  1.000
примітка_інфаркт_стент   0.041  0.200  0.000  1.000
тривалість_z            -0.000  0.975 -1.215  4.608
ліжко_лог_z             -0.000  0.996 -3.143  2.528
ПКТ_виміряно             0.380  0.487  0.000  1.000
ПКТ_≤0.1                 0.273  0.447  0.000  1.000
ПКТ_лог_z               -0.000  1.000 -1.398  7.189
білок_z                 -0.000  1.000 -2.023  2.570
гематокрит_частка        0.410  0.069  0.160  0.570
гематокрит_пропуск       0.165  0.373  0.000  1.000
```

**Перед нормалізацією** обов'язково перевірте варіативність. Стовпець, у якому всі значення однакові, дасть нуль у знаменнику — і мовчазний `NaN` замість помилки.

```
for c in МОЇ:
    з = pd.to_numeric(df[c], errors='coerce').dropna()
    if len(з) and з.nunique() == 1:
        print(f'УВАГА: {c} — сталий стовпець, нормалізація дасть
ділення на нуль')
    elif len(з) and з.max() == з.min():
        print(f'УВАГА: {c} — max дорівнює min')
print('Перевірку варіативності виконано.')

Перевірку варіативності виконано.

# --- САМОПЕРЕВІРКА 2 ---
assert all(len(ПІДГОТОВКА[c].strip()) >= 15 for c in МОЇ), \
    'Для кожного стовпця вкажіть спосіб підготовки'
assert len(БЕЗГЛУЗДІ) >= 3, 'Етап 3 не завершено'
print('Самоперевірка 2 пройдена.')

Самоперевірка 2 пройдена.
```

---

# Етап 6. Виправлення чужого аналізу

Відкрийте файл **`ЛР3_помилковий_аналіз.ipynb`**. Це звіт, який виконується без жодної помилки й містить десять хибних місць.

Для кожного з десяти заповніть рядок нижче:

- **розділ** — номер розділу звіту;
- **що не так** — яка шкала порушена або яке припущення хибне;
- **чому число неправильне** — що саме воно означає насправді;
- **як правильно** — коректний варіант або обґрунтована відмова від показника.

```
ВИПРАВЛЕННЯ = [
    (1, 'Номінальна шкала (бінарна) пораховано як кількісну:
«помер/жив» кодовано 1 і 2, обчислено середнє 1.50.',
        'Коди 1 і 2 — довільні ярлики категорій. Середнє 1.4959 — це
лише 1 + частка тих, хто вижив; пацієнта «посередині між летальним і
сприятливим» не існує.',
        'Для номінальної змінної — частоти: 61 помер (50.4 %), 60
вижили (49.6 %); показник — летальність 50.4 %. Середнє відхилити.'),
    (2, 'Дані непридатні: одна категорія записана по-різному («ж» і
«жін», «ч» і «чол»), через що nunique() = 4.',
        'Чотири «статі» — це артефакт введення: насправді дві
категорії. Тому «найбільша група 56 осіб, 46 %» хибна: жінок 75 (62
%).',
        'Спершу звести написання до одного коду, потім рахувати: жінки
75 (62.0 %), чоловіки 46 (38.0 %).'),
    (3, 'Порядкова шкала: «ступінь КТ» відсортовано алфавітом
(sort_index), а «середній ступінь» пораховано як середнє кодів; плюс
12 написань замість 5 і 65 % пропусків.',
        'Алфавіт дає «найлегший — важкий, найважчий — середній» —
безглуздо; середнє кодів залежить від довільних кодів, а 5.9 →
«легкий» просто збіг індексів.',
        'Нормалізувати написання, задати порядок вручну (легкий <
середній < важкий < вкрай важкий < критичний); показати медіану й
квартилі, а не середнє.'),
    (4, 'Дані непридатні: to_numeric(errors="coerce") мовчки відкинув
16 текстових записів («30-40%», «до 17%», «60-70») із 43 заповнених.',
        'Середнє 39.6 % пораховано лише на 27 із 43 записів (37 %
втрачено), причому відкинуто саме діапазонні записи. Значення 39.6 %
не є «середнім обсягом ураження».',
        'Розібрати діапазони (середина інтервалу; «до 17%» як 0–17) і
рахувати на всіх 43; обов’язково вказати, що КТ є лише у 36 %
пацієнтів (n = 43 із 121).'),
    (5, 'Дані непридатні: у «spO2 без дот» є значення 0.9 — це 90 %,
записане часткою; дві несумісні кодування одного стовпця.',
        'Одна точка 0.9 тягне середнє вниз (82.05 замість 82.79) і
спотворює min; фраза «у більшості пацієнтів виражена дихальна
недостатність» з середнього не випливає — середнє не каже про
більшість.',
        'Виправити 0.9 → 90 (або виключити), описувати медіаною (84) і
часткою пацієнтів нижче порогу (75 % — 90 із 120 — мали spO2 < 90
%).'),
    (6, 'Інтервальна шкала: коефіцієнт варіації обчислено для дат (дні
від 1 січня 1970).',
        'CV = SD / середнє, а середнє дат залежить від умовного нуля:
при відліку від 1970 вийшло 0.595 %, при відліку від 2000 — 1.4 %. Це
```

```
число показує, як далеко 1970 рік, а не однорідність потоку.',
        'CV для дат не визначений. Розкид описують у днях (SD ≈ 113
днів) або беруть лічильник (відношень) — кількість госпіталізацій на
тиждень — і його CV (≈ 126 %: потік вкрай нерівномірний).'),
    (7, 'Номінальна × номінальна: Пірсон між кодами статі (ч=0, ж=1) і
летальності; висновок «зв’язку нема».',
        'r = 0.177 не «близьке до нуля»: χ² = 3.78, p = 0.052 — на
межі значущості; летальність жінок 57 %, чоловіків 39 %. Знак r
залежить від довільного кодування, а «відсутність лінійного зв’язку»
для категорій не має змісту.',
        'Використати χ² та V Крамера (V = 0.177) із таблицею
спряженості; висновок: зв’язок слабкий, на межі значущості, а не
відсутній.'),
    (8, 'Нормалізацію (x − min)/(max − min) застосовано до всіх
стовпців типу number без урахування шкали й варіативності.',
        'ФЕРИТ2 сталий (єдине значення 1265): 0/0 дає повний стовпець
NaN, тож «усі стовпці приведено до [0,1]» неправда; нормалізовано й
номінальну «помер/жив»; одна дивна точка (spO2 = 0.9) стискає решту.',
        'Відбирати стовпці за шкалою, перед нормалізацією перевіряти
max ≠ min і мінімальну кількість значень, виправляти викиди; параметри
рахувати лише на навчальній вибірці.'),
    (9, 'Дані непридатні: dropna() без аналізу того, що означають
пропуски; у звіті стверджується протилежне до результату.',
        'Після dropna() лишилося 0 рядків із 121 (звіт друкує «Стало
рядків: 0» і все одно називає набір повністю заповненим). Пропуск
буває «не вимірювали» (ФЕРИТ2 — 120 із 121) і «нема» (дата розтину
порожня для всіх 60 живих).',
        'Видалити розріджені стовпці, а для решти вирішувати окремо:
прапорець пропуску, медіана (на train) або «нема». Наприклад, стовпці
з ≤ 20 % пропусків дають 65 повних рядків.'),
    (10, 'Витік цілі: «Дата розтину» існує лише в померлих і сама є
наслідком результату; NaT перетворено на int64 (−9.2·10¹⁸); крім того
кореляції пораховано на n = 6.',
        'r = 1.00 — не «найсильніший предиктор», а ознака, що стовпець
випливає з цілі; інтерлейкін-6 (ІЛ6-2) має r = 0.62 лише на 6
пацієнтах; дати рахуються як числа, а |r| ховає знак.',
        'Виключити ознаки, відомі лише після результату; брати стовпці
з достатнім n (≥ 60), показувати n, знак r і p; для дат — тривалості.
Перерахований рейтинг — нижче.'),
]

assert len(ВИПРАВЛЕННЯ) == 10, f'Знайдено {len(ВИПРАВЛЕННЯ)} помилок
із 10'
for p in ВИПРАВЛЕННЯ:
    assert len(p) == 4 and all(str(x).strip() for x in p), 'Заповніть
усі чотири поля'
    assert len(str(p[2])) >= 30 and len(str(p[3])) >= 25, 'Пояснення
закороткі'
```

```
показати(pd.DataFrame(ВИПРАВЛЕННЯ, columns=['Розділ', 'Що не так',
'Чому число неправильне', 'Як правильно']))
```

```
<pandas.io.formats.style.Styler at 0x1fbc7577390>
```

## Виправте щонайменше три помилки в коді

Оберіть три з десяти та покажіть правильне обчислення поруч із неправильним.

```
# Правильні обчислення поруч із хибними (розділи 2, 3, 4, 6, 9, 10)

print('=' * 70); print('РОЗДІЛ 2 — Стать'); print('=' * 70)
print('Хибно (сирі написання):');
print(df['Стать'].value_counts().to_string())
стать_н = df['Стать'].str.strip().str.lower().str[0].map({'ж':
'жінки', 'ч': 'чоловіки'})
print('\nПравильно (зведено до двох категорій):')
print(pd.DataFrame({'осіб': стать_н.value_counts(), '%': (100 *
стать_н.value_counts(normalize=True)).round(1)}).to_string())

print(); print('=' * 70); print('РОЗДІЛ 3 — ступінь КТ'); print('=' *
70)
ПОРЯДОК_КТ = ['легкий', 'середній', 'важкий', 'вкрай важкий',
'критичний']
виправ = {'сердній': 'середній', 'серед ступ': 'середній', 'серед
ступеня': 'середній', 'легка': 'легкий'}
ст = df['ступінь КТ'].str.strip().str.lower().replace(виправ)
ст = pd.Series(pd.Categorical(ст, categories=ПОРЯДОК_КТ,
ordered=True), index=df.index)
к = np.sort(ст.cat.codes[ст.notna()].values)
print('Хибно: за зростанням «найлегший — важкий, найважчий — середній»
(алфавіт), середнє кодів 5.9 -> «легкий».')
print('Правильно (заданий порядок), частоти:');
print(ст.value_counts().sort_index().to_string())
print('Медіана:', ПОРЯДОК_КТ[к[(len(к) - 1) // 2]], '| Q1 .. Q3:',
ПОРЯДОК_КТ[к[int(np.ceil(.25 * len(к))) - 1]], '..',
ПОРЯДОК_КТ[к[int(np.ceil(.75 * len(к))) - 1]],
      '| n =', len(к), 'із', len(df), '(пропусків',
int(ст.isna().sum()), ')')

print(); print('=' * 70); print('РОЗДІЛ 4 — обсяг ураження (КТ)');
print('=' * 70)
def розібрати_кт(v):
    if isinstance(v, (int, float)):
        return 100 * v if pd.notna(v) else np.nan
    числа = [float(x.replace(',', '.')) for x in re.findall(r'\d+(?:
[.,]\d+)?', v)]
    if not числа: return np.nan
    return числа[0] / 2 if 'до' in v else float(np.mean(числа))
```

```
кт_всі = df['КТ'].map(розібрати_кт)
кт_хибно = 100 * pd.to_numeric(df['КТ'], errors='coerce')
print(f'Хибно : n = {кт_хибно.notna().sum()}, середнє
{кт_хибно.mean():.1f} %, медіана {кт_хибно.median():.1f} %, SD
{кт_хибно.std():.1f} %')
print(f'Правильно: n = {кт_всі.notna().sum()}, середнє
{кт_всі.mean():.1f} %, медіана {кт_всі.median():.1f} %, SD
{кт_всі.std():.1f} %')
print(f'(КТ є лише у {кт_всі.notna().sum()} із {len(df)} пацієнтів =
{100 * кт_всі.notna().mean():.0f} %: висновок стосується лише
обстежених)')

======================================================================
РОЗДІЛ 2 — Стать
======================================================================
Хибно (сирі написання):
Стать
ж      56
ч      34
жін    19
чол    12

Правильно (зведено до двох категорій):
          осіб     %
Стать
жінки       75  62.0
чоловіки    46  38.0

======================================================================
РОЗДІЛ 3 — ступінь КТ
======================================================================
Хибно: за зростанням «найлегший — важкий, найважчий — середній»
(алфавіт), середнє кодів 5.9 -> «легкий».
Правильно (заданий порядок), частоти:
легкий           7
середній        18
важкий          10
вкрай важкий     4
критичний        3
Медіана: середній | Q1 .. Q3: середній .. важкий | n = 42 із 121
(пропусків 79 )

======================================================================
РОЗДІЛ 4 — обсяг ураження (КТ)
======================================================================
Хибно : n = 27, середнє 39.6 %, медіана 30.0 %, SD 22.9 %
Правильно: n = 43, середнє 42.3 %, медіана 35.0 %, SD 22.2 %
(КТ є лише у 43 із 121 пацієнтів = 36 %: висновок стосується лише
обстежених)
```

# Етап 7. Висновки та індивідуальне питання

```
# Підготовка до відповіді: вікові групи
вік_ = df['Вік']
групи = pd.cut(вік_, bins=[0, 39, 60, 200], labels=['до 40', '40–60',
'понад 60'])
к_груп = pd.Series(групи.cat.codes, index=df.index)           # 0, 1,
2 – порядкові коди
розподіл = групи.value_counts().sort_index()
print(розподіл.to_string())
print()
print('Межі груп: 39 -> «до 40», 40 -> «40–60», 60 -> «40–60», 61 ->
«понад 60»')
print('Вік 41 і 59 -> одна група:', групи[вік_.isin([41,
59])].unique().tolist(),
      '| вік 59 і 61 -> різні групи:', групи[вік_.isin([59,
61])].unique().tolist())

m = pd.concat([вік_, к_груп, сат], axis=1).dropna()
m.columns = ['вік', 'група', 'spO2']
r_вік = stats.pearsonr(m['вік'], m['spO2'])[0]
r_група = stats.spearmanr(m['група'], m['spO2'])[0]
r_вік_c = stats.spearmanr(m['вік'], m['spO2'])[0]
print()
print(f'Зв’язок зі spO2 (Спірмен): за точним віком ρ = {r_вік_c:.3f};
за віковою групою ρ = {r_група:.3f}')
середні = m.groupby('група')['spO2'].agg(['count', 'median']).round(1)
середні.index = ['до 40', '40–60', 'понад 60']
print(середні.to_string())
```

```
Вік
до 40        3
40–60       37
понад 60    81

Межі груп: 39 -> «до 40», 40 -> «40–60», 60 -> «40–60», 61 -> «понад
60»
Вік 41 і 59 -> одна група: ['40–60'] | вік 59 і 61 -> різні групи:
['40–60', 'понад 60']

Зв’язок зі spO2 (Спірмен): за точним віком ρ = -0.342; за віковою
групою ρ = -0.303
          count  median
до 40         3    91.0
40–60        37    89.0
понад 60     80    83.0
```

```
_пкт = df['Прокальцитонін'].dropna()
ВИСНОВКИ = [
    f'Прокальцитонін заповнено лише у {len(_пкт)} із {len(df)}
пацієнтів ({100 * len(_пкт) / len(df):.0f} %), і {int((_пкт ==
0.1).sum())} із цих значень дорівнюють 0.1 — '
    f'нижній межі чутливості тесту; тому середнє {_пкт.mean():.2f}
(вище за {100 * (_пкт < _пкт.mean()).mean():.0f} % спостережень)
описує тест, а не пацієнтів, і для цього стовпця слід брати медіану
{_пкт.median():.1f} та частку значень вище порогу.',
    'У стовпці «Виписка» дві клітинки («24.024.2021», «20.24.2021») не
є датами; їх вдалося відновити як Поступлення + Ліжко днів, і дати
збіглися з датами розтину обох померлих, '
    'а ще п’ять записів мають суперечність між датами й «Ліжко днів»
понад 3 дні — тому тривалість стаціонарного лікування безпечніше брати
з однієї перевіреної ознаки, не змішуючи обидві.',
    f'Для пари «Вік × spO2 без дот» один запис 0.9 (насправді 90 %)
тягне кореляцію до r = {пірсон(вік, сат_сира)[1]:.2f} (p =
{пірсон(вік, сат_сира)[2]:.2f}); після виправлення r = {r_осн:.2f} (p
= {p_осн:.3f}) — '
    'слабкий-помірний негативний зв’язок; одна кодова помилка здатна
змінити висновок із «зв’язку нема» на «зв’язок є».',
    'ЛН — порядкова шкала з шістьма ступенями, серед яких проміжні
(«II-III» найчастіший: 59 із 121), а «III-IV» записано латинським IV;
середнє для неї обчислювати не можна, '
    'а упорядковувати треба вручну, бо автоматичне сортування
збігається з клінічним порядком лише випадково.',
]
assert len(ВИСНОВКИ) >= 3 and all(len(в) >= 60 for в in ВИСНОВКИ), \
    'Потрібні три змістовні висновки'
for і, в in enumerate(ВИСНОВКИ, 1):
    print(f'{і}. {в}')
    print()
```

```
1. Прокальцитонін заповнено лише у 46 із 121 пацієнтів (38 %), і 30 із
цих значень дорівнюють 0.1 — нижній межі чутливості тесту; тому
середнє 0.75 (вище за 93 % спостережень) описує тест, а не пацієнтів,
і для цього стовпця слід брати медіану 0.1 та частку значень вище
порогу.

2. У стовпці «Виписка» дві клітинки («24.024.2021», «20.24.2021») не є
датами; їх вдалося відновити як Поступлення + Ліжко днів, і дати
збіглися з датами розтину обох померлих, а ще п’ять записів мають
суперечність між датами й «Ліжко днів» понад 3 дні — тому тривалість
стаціонарного лікування безпечніше брати з однієї перевіреної ознаки,
не змішуючи обидві.

3. Для пари «Вік × spO2 без дот» один запис 0.9 (насправді 90 %) тягне
кореляцію до r = -0.15 (p = 0.11); після виправлення r = -0.27 (p =
0.003) — слабкий-помірний негативний зв’язок; одна кодова помилка
здатна змінити висновок із «зв’язку нема» на «зв’язок є».
```

4. ЛН — порядкова шкала з шістьма ступенями, серед яких проміжні («II-III» найчастіший: 59 із 121), а «III-IV» записано латинським IV; середнє для неї обчислювати не можна, а упорядковувати треба вручну, бо автоматичне сортування збігається з клінічним порядком лише випадково.

```
print('ВАШЕ ПИТАННЯ:'); print(B['питання']); print()
ВІДПОВІДЬ = (
    'До перетворення «Вік» — шкала відношень: це повні роки від природного нуля, різниці й відношення мають зміст (60 років удвічі більше за 30), '
    'допустимі середнє, SD, CV і кореляція Пірсона. Після розбиття на групи «до 40», «40–60», «понад 60» вийшла порядкова шкала: групи впорядковані, '
    'але відстань між ними невідома, а всередині групи різниці стерті. '
    f'ВТРАТИЛИ: точність і метричність — пацієнти 41 і 59 років тепер ідентичні, а 59 і 61 — різні, хоча різниця менша; зникли середнє й Пірсон, лишилися медіана й Спірмен; '
    f'розподіл нерівний (до 40 років лише {int(розподіл.iloc[0])} пацієнти, понад 60 — {int(розподіл.iloc[2])}), тож перша група статистично майже порожня; '
    f'зв’язок зі spO2 виглядає слабшим (Спірмен: за точним віком ρ = {r_вік_c:.2f}, за групою ρ = {r_група:.2f}), бо частину інформації відкинуто; результат залежить від довільних меж (40 і 60 — до якої групи належать?). '
    'ЗДОБУЛИ: стійкість до помилок і викидів у віці, зрозумілі клінічні категорії для звіту, можливість порівнювати групи без припущення лінійності '
    '(медіана spO2 у групах), і придатність до моделей із категоріальними вхідними даними. '
    'Висновок: перетворення виправдане для презентації та клінічного поділу, але для кореляційного аналізу й моделі краще лишити точний вік — з нього завжди можна отримати групи, а навпаки не можна.'
)
assert len(ВІДПОВІДЬ.strip()) >= 200, 'Відповідь має бути розгорнутою (від 200 символів)'
print(ВІДПОВІДЬ)
```

```
ВАШЕ ПИТАННЯ:
Стовпець «Вік» можна перетворити на вікові групи «до 40», «40–60», «понад 60». Яка шкала була до перетворення і яка стала? Що ви при цьому втратили і що здобули?

До перетворення «Вік» — шкала відношень: це повні роки від природного нуля, різниці й відношення мають зміст (60 років удвічі більше за 30), допустимі середнє, SD, CV і кореляція Пірсона. Після розбиття на групи «до 40», «40–60», «понад 60» вийшла порядкова шкала: групи
```

```
впорядковані, але відстань між ними невідома, а всередині групи різниці стерті. ВТРАТИЛИ: точність і метричність — пацієнти 41 і 59 років тепер ідентичні, а 59 і 61 — різні, хоча різниця менша; зникли середнє й Пірсон, лишилися медіана й Спірмен; розподіл нерівний (до 40 років лише 3 пацієнти, понад 60 — 81), тож перша група статистично майже порожня; зв’язок зі spO2 виглядає слабшим (Спірмен: за точним віком ρ = -0.34, за групою ρ = -0.30), бо частину інформації відкинуто; результат залежить від довільних меж (40 і 60 — до якої групи належать?). ЗДОБУЛИ: стійкість до помилок і викидів у віці, зрозумілі клінічні категорії для звіту, можливість порівнювати групи без припущення лінійності (медіана spO2 у групах), і придатність до моделей із категоріальними вхідними даними. Висновок: перетворення виправдане для презентації та клінічного поділу, але для кореляційного аналізу й моделі краще лишити точний вік — з нього завжди можна отримати групи, а навпаки не можна.
```

---

# Етап 8. Декларація про використання інструментів ШІ

Користуватися мовними моделями дозволено. Це нормальна інженерна практика.

**Обов'язковою є прозорість:** ви вказуєте, де саме і як скористалися інструментом — так само, як у науковій статті вказують використане програмне забезпечення. Задекларована допомога порушенням не є.

Порушенням є інше: здати роботу, яку ви не можете пояснити. На захисті вас попросять показати конкретний рядок свого коду, пояснити, чому він саме такий, і внести в нього невелику зміну на місці.

Для кожного етапу оберіть одне з формулювань:

- `'ні'` — інструментами ШІ не користувався
- `'синтаксис'` — питав про назву функції або синтаксис, код писав сам
- `'код, перевірено'` — код згенеровано, я його прочитав, перевірив і розумію
- `'текст, перероблено'` — згенеровано чернетку тексту, яку я переписав по суті
- `'текст, як є'` — згенерований текст залишено майже без змін

```
ДЕКЛАРАЦІЯ = {
    'етап 1 — паспорт змінних':        'текст, перероблено',
    'етап 2 — допустимі статистики':   'код, перевірено',
    'етап 3 — безглузді числа':        'код, перевірено',
    'етап 4 — міра зв’язку':           'код, перевірено',
    'етап 5 — підготовка до моделі':   'код, перевірено',
    'етап 6 — виправлення аналізу':    'текст, перероблено',
    'етап 7 — висновки й відповідь':   'текст, перероблено',
```

```
}

ЩО_САМЕ_Я_ПЕРЕВІРИВ = (
    'Ноутбук виконується згори донизу на чистому ядрі; '
    'числа звірено повторним обчисленням; зіпсовані дати виписки
відновлено та підтверджено датами розтину; кожен із десяти дефектів
звіту відтворено в даних.'
)
ПІДТВЕРДЖУЮ = True      # я можу пояснити кожен рядок свого коду
                        # і внести в нього зміни на вимогу під час
захисту

# --- ФІНАЛЬНА САМОПЕРЕВІРКА ---
ФОРМУЛЮВАННЯ = {'ні', 'синтаксис', 'код, перевірено', 'текст,
перероблено', 'текст, як є'}

перевірки = {
    'заповнено титульні поля':        bool(ПРІЗВИЩЕ_ІМЯ and ГРУПА and
НОМЕР),
    'етап 1: паспорт із 8 стовпців':  all(ПАСПОРТ[c][0] and ПАСПОРТ[c]
[1] for c in МОЇ),
    'етап 3: три безглузді числа':    len(БЕЗГЛУЗДІ) >= 3,
    'етап 4: міру обрано й пояснено': bool(ОБРАНА_МІРА and
len(ЧОМУ_САМЕ_ВОНА) >= 40),
    'етап 5: підготовку описано':     all(len(ПІДГОТОВКА[c].strip())
>= 15 for c in МОЇ),
    'етап 6: знайдено 10 помилок':    len(ВИПРАВЛЕННЯ) == 10,
    'етап 7: три висновки':           len(ВИСНОВКИ) >= 3,
    'етап 7: відповідь на питання':   len(ВІДПОВІДЬ.strip()) >= 200,
    'декларацію заповнено':           all(v in ФОРМУЛЮВАННЯ for v in
ДЕКЛАРАЦІЯ.values()),
    'опис перевірки ≥ 120 символів':  len(ЩО_САМЕ_Я_ПЕРЕВІРИВ.strip())
>= 120,
    'підтвердження авторства':        ПІДТВЕРДЖУЮ is True,
}

print('ЧЕК-ЛИСТ ГОТОВНОСТІ')
print('=' * 52)
for назва, ок in перевірки.items():
    print(('  ГОТОВО     ' if ок else '  НЕ ГОТОВО ') + назва)
print('=' * 52)

assert all(перевірки.values()), 'Робота не готова до здачі — див. чек-
лист вище'
print()
print(f'Робота варіанта {В["варіант"]} готова до захисту.')
print(f'{ПРІЗВИЩЕ_ІМЯ}, {ГРУПА}')
```

```
ЧЕК-ЛИСТ ГОТОВНОСТІ
====================================================
```

```
  ГОТОВО     заповнено титульні поля
  ГОТОВО     етап 1: паспорт із 8 стовпців
  ГОТОВО     етап 3: три безглузді числа
  ГОТОВО     етап 4: міру обрано й пояснено
  ГОТОВО     етап 5: підготовку описано
  ГОТОВО     етап 6: знайдено 10 помилок
  ГОТОВО     етап 7: три висновки
  ГОТОВО     етап 7: відповідь на питання
  ГОТОВО     декларацію заповнено
  ГОТОВО     опис перевірки ≥ 120 символів
  ГОТОВО     підтвердження авторства
==================================================

Робота варіанта 11 готова до захисту.
Перхун Максим, ШІ-34
```

---

## Перед здачею

Виконайте **Kernel → Restart & Run All**. Ноутбук має пройти згори донизу без жодної помилки — це перше, що перевіряється на захисті.